

NAME: VADDEM CHANDANA

HALL TICKET NO: 2403A51186

BATCH NO: 09

ASSIGNMENT 12.1

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
NS_2 (Mounika)			
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week6 - Monday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber:12.1(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms Lab Objectives: Apply AI-assisted programming to implement and optimize	Week6 - Monday	

NAME: VADDEM CHANDANA

HALL TICKET NO: 2403A51186

BATCH NO: 09

ASSIGNMENT 12.1

	sorting and searching algorithms. • Compare different algorithms in terms of efficiency and use cases. • Understand how AI tools can suggest optimized code and complexity improvements.	
	Task Description #1 (Sorting – Merge Sort Implementation) • Task: Use AI to generate a Python program that implements the Merge Sort algorithm. • Instructions: ◦ Prompt AI to create a function <code>merge_sort(arr)</code> that sorts a list in ascending order. ◦ Ask AI to include time complexity and space complexity in the function docstring. ◦ Verify the generated code with test cases. Code: <pre>[3] ✓ 0s def merge_sort(arr): if len(arr) <= 1: return arr mid = len(arr) // 2 left_half = arr[:mid] right_half = arr[mid:] left_half = merge_sort(left_half) right_half = merge_sort(right_half) return merge(left_half, right_half) def merge(left, right): merged_list = [] i = j = 0 while i < len(left) and j < len(right): if left[i] < right[j]: merged_list.append(left[i]) i += 1 else: merged_list.append(right[j]) j += 1 while i < len(left):</pre> Toggle Gemini	

NAME: VADDEM CHANDANA

HALL TICKET NO: 2403A51186

BATCH NO: 09

ASSIGNMENT 12.1

```
while i < len(left):
    merged_list.append(left[i])
    i += 1

while j < len(right):
    merged_list.append(right[j])
    j += 1

return merged_list

# Test cases
test_cases = [
    ([3, 6, 8, 10, 1, 2, 1], [1, 1, 2, 3, 6, 8, 10]),
    ([5, 4, 3, 2, 1], [1, 2, 3, 4, 5]),
    ([1], [1]),
    ([], []),
    ([1, 2, 3, 4, 5], [1, 2, 3, 4, 5]),
    ([5, 1, 4, 2, 8], [1, 2, 4, 5, 8])
]

for input_arr, expected_output in test_cases:
    actual_output = merge_sort(input_arr.copy()) # Use a copy to avoid modifying the original list
    assert actual_output == expected_output, f"Input: {input_arr}, Expected: {expected_output}, Got: {actual_output}"
    print(f"Test case passed for input: {input_arr}")
```

- Expected Output:

- A functional Python script implementing Merge Sort with proper documentation.

Output:

```
➡ Test case passed for input: [3, 6, 8, 10, 1, 2, 1]
Test case passed for input: [5, 4, 3, 2, 1]
Test case passed for input: [1]
Test case passed for input: []
Test case passed for input: [1, 2, 3, 4, 5]
Test case passed for input: [5, 1, 4, 2, 8]
```

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
 - Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.
 - Include docstrings explaining best, average, and worst-case complexities.
 - Test with various inputs.
- Expected Output:
 - Python code implementing binary search with AI-generated comments and docstrings.

Code and output:

NAME: VADDEM CHANDANA

HALL TICKET NO: 2403A51186

BATCH NO: 09

ASSIGNMENT 12.1

```
def binary_search(arr, target):
    """
    Finds the index of a target element in a sorted list using binary search.
    Time Complexity: O(log n)
    Space Complexity: O(1)
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

# Test cases
test_cases = [
    ([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5, 4), # Target in the middle
    ([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 1, 0), # Target at the beginning
    ([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 10, 9), # Target at the end
    ([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 11, -1), # Target not in the list (greater than max)
```

```
([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 10, 9), # Target at the end
([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 11, -1), # Target not in the list (greater than max)
([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 0, -1), # Target not in the list (less than min)
([], 5, -1), # Empty list
([5], 5, 0), # Single element list, target is the element
([5], 1, -1), # Single element list, target is not the element
([1, 3, 5, 7, 9], 3, 1), # Odd number of elements
([1, 3, 5, 7, 9], 6, -1) # Odd number of elements, target not in list
]

for input_arr, target, expected_output in test_cases:
    actual_output = binary_search(input_arr, target)
    assert actual_output == expected_output, f"Input: {input_arr}, Target: {target}, Expected: {expected_output}, Got: {actual_output}"
    print(f"Test case passed for input: {input_arr}, Target: {target}")

Test case passed for input: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Target: 5
Test case passed for input: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Target: 1
Test case passed for input: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Target: 10
Test case passed for input: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Target: 11
Test case passed for input: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Target: 0
Test case passed for input: [], Target: 5
Test case passed for input: [5], Target: 5
Test case passed for input: [5], Target: 1
Test case passed for input: [1, 3, 5, 7, 9], Target: 3
Test case passed for input: [1, 3, 5, 7, 9], Target: 6
```

Task Description #3 (Real-Time Application – Inventory Management System)

- Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:
 - Quickly search for a product by ID or name.
 - Sort products by price or quantity for stock analysis.
- Task:
 - Use AI to suggest the most efficient search and sort algorithms for this use case.
 - Implement the recommended algorithms in Python.
 - Justify the choice based on dataset size, update frequency, and performance requirements.

NAME: VADDEM CHANDANA
HALL TICKET NO: 2403A51186
BATCH NO: 09
ASSIGNMENT 12.1

Code:

```
import pandas as pd

# 1. Explain Algorithm Choices
data = {
    "Algorithm Type": ["Search", "Sort"],
    "Recommended Algorithm": ["Binary Search", "Merge Sort or Tim Sort"],
    "Justification (Dataset Size, Update Frequency, Performance)": [
        "For efficient searching of a *sorted* inventory (e.g., by product ID or price)",
        "Suitable for sorting the inventory for display purposes (e.g., by price, popularity)"
    ],
    "Time Complexity (Average Case)": ["O(log n)", "O(n log n)"]
}

df = pd.DataFrame(data)

display(df)

# 2. Implement Search Algorithm
def binary_search(arr, target):
    """
    Finds the index of a target element in a sorted list using binary search.
    Time Complexity: O(log n)
    Space Complexity: O(1)
    """
    left, right = 0, len(arr) - 1
```

```
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

# 3. Implement Sort Algorithm (Using Python's built-in Tim Sort for efficiency)
def sort_inventory(arr, key=None, reverse=False):
    """
    Sorts the inventory list using Python's built-in Tim Sort.

    Args:
        arr: The list of inventory items (e.g., dictionaries).
        key: A function to specify the sorting key (e.g., lambda item: item['price']).
        reverse: If True, sort in descending order.

    Returns:
        A new list containing the sorted inventory items.
    """
```

- Expected Output:
 - A table mapping operation → recommended algorithm → justification.
 - Working Python functions for searching and sorting the inventory.

NAME: VADDEM CHANDANA

HALL TICKET NO: 2403A51186

BATCH NO: 09

ASSIGNMENT 12.1

Output:

```
return sorted(arr, key=key, reverse=reverse)
```

	Algorithm Type	Recommended Algorithm	Justification (Dataset Size, Update Frequency, Performance)	Time Complexity (Average Case)	
0	Search	Binary Search	For efficient searching of a "sorted" inventor...	$O(\log n)$	
1	Sort	Merge Sort or Tim Sort	Suitable for sorting the inventory for display...	$O(n \log n)$	

- ☒ Deliverables (For All Tasks)
- 1. AI-generated prompts for code and test case generation.
 - 2. At least 3 assert test cases for each task.
 - 3. AI-generated initial code and execution screenshots.
 - 4. Analysis of whether code passes all tests.
 - 5. Improved final version with inline comments and explanation.
 - 6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.